Task Manager Assessment Submission

Candidate	Shih-En Chen
Repository	https://github.com/jackychen21/pulseboard
Live Demo	https://pulseboard-red.vercel.app
Stack	React, TypeScript, Vite, Supabase, anonymous auth

1. Solution Overview

PulseBoard is a responsive four-column Kanban board where guest users can create tasks, move them between columns, and organize their work. The interface is designed to be simple, clean, and easy to use, with clear feedback for loading, empty, and error states.

2. Design Decisions

- Designed a modern dark interface with warm accent colors to make the board feel clean and engaging.
- Used the native HTML5 Drag and Drop API to keep interactions simple without adding extra libraries.
- Added summary cards, due date badges, and filters to help users organize and track tasks more easily.
- Included a demo mode so the app can still run without Supabase credentials.

3. Features Implemented

Core features (required):

- Anonymous guest session, RLS isolation, task creation, drag-and-drop status changes, loading/error states

Advanced features (optional):

- Search & filtering — matches task title/description against the search query, combined with priority and due-date-window filters
- Due date urgency indicators — `getDueBadge` computes days remaining and assigns a neutral/warning/danger tone (overdue = red, due within 2 days = yellow)
- Board summary/stats — aggregates total, completed, in-flight, overdue, and due-soon counts from the same `getDueBadge` logic, shown in the sidebar

4. Database Schema

The full SQL setup is included in supabase/schema.sql. The main tasks table is summarized below.

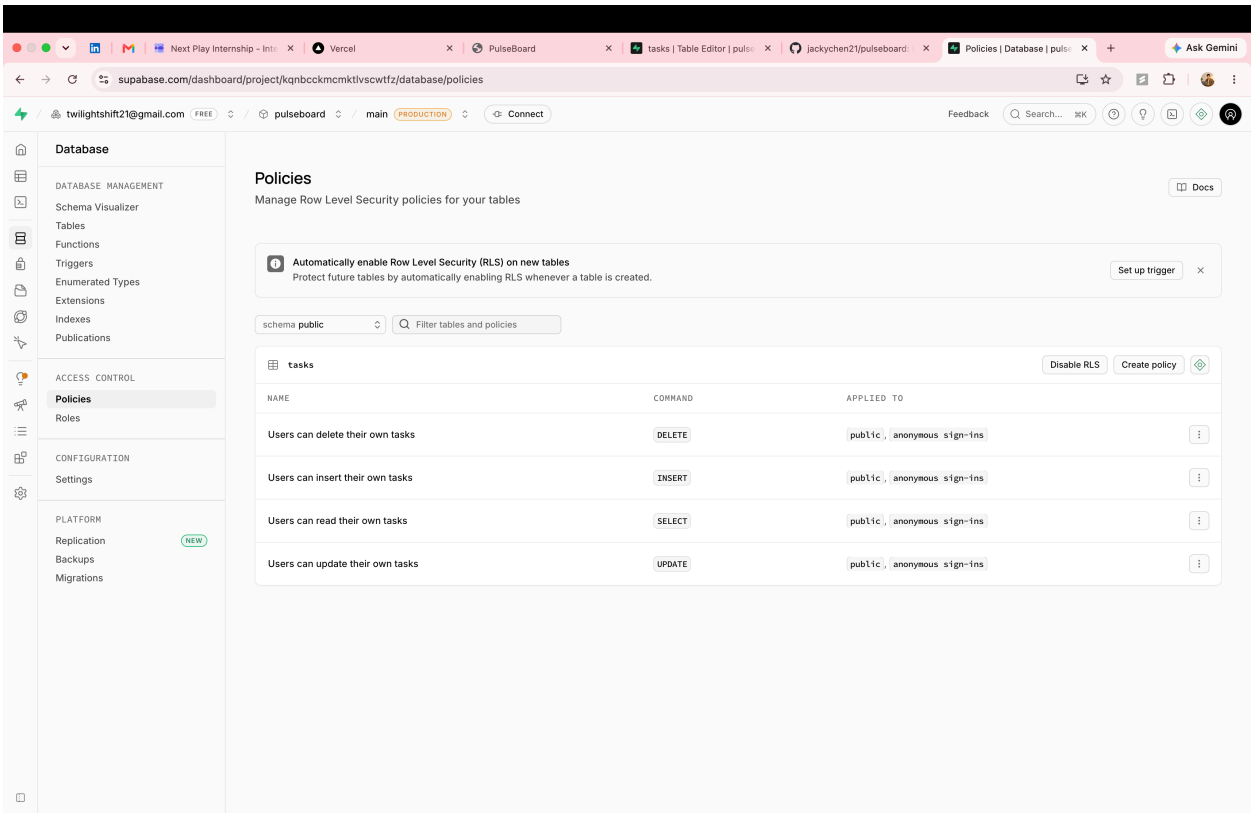
Field	Type	Notes
id	uuid	Primary key
title	text	Required
status	text	todo, in_progress, in_review, done

description	text	Optional detail body, defaults to empty string
priority	text	low, normal, high
user_id	uuid	Tied to anonymous guest session via auth.uid()

Additional field included in the actual schema: created_at timestamptz default timezone('utc', now()).

Full SQL (supabase/schema.sql)

```
create extension if not exists "pgcrypto";

create table if not exists public.tasks (
  id uuid primary key default gen_random_uuid(),
  title text not null,
  description text not null default '',
  status text not null default 'todo' check (status in ('todo',
'in_progress', 'in_review', 'done')),
  priority text not null default 'normal' check (priority in ('low',
'normal', 'high')),
  due_date date,
  user_id uuid not null default auth.uid(),
  created_at timestamptz not null default timezone('utc', now())
);

alter table public.tasks enable row level security;

create policy "Users can read their own tasks"
on public.tasks
for select
using (auth.uid() = user_id);

create policy "Users can insert their own tasks"
on public.tasks
for insert
with check (auth.uid() = user_id);

create policy "Users can update their own tasks"
on public.tasks
for update
using (auth.uid() = user_id)
with check (auth.uid() = user_id);

create policy "Users can delete their own tasks"
on public.tasks
for delete
using (auth.uid() = user_id);

alter publication supabase_realtime add table public.tasks;
```



Screenshot of Supabase RLS Policies (Database → Policies)

5. Setup Instructions

- Install dependencies with pnpm install.
- Copy .env.example to .env and set VITE_SUPABASE_URL plus VITE_SUPABASE_ANON_KEY.
- Enable Anonymous sign-in in Supabase Auth.
- Run the SQL in supabase/schema.sql.
- Start locally with pnpm dev.
- Deploy the Vite app to Vercel, Netlify, or Cloudflare Pages with the same two environment variables.

6. Tradeoffs and Next Steps

- I prioritized a stable, reviewer-friendly core board over adding a larger optional backend surface.
- If I had more time, I would add assignees, task comments, and a dedicated activity log.
- With more time, I would add more tests for authentication and drag-and-drop functionality.